

Web Crawler Part 2: Scaling Design

The web crawler created and hosted on AWS for part 1 of this assignment can be demoed here: <https://edabpk62tn.us-east-1.awsapprunner.com/docs>

At its core, my crawler consists of several Python files that define functions to crawl a URL and return metadata. These include an init file specifying the crawler package to import, a function to fetch HTTP content, a function for safety checks (prevents internal redirects and private domains), a parser to extract structured metadata from the HTTP result, a classifier to determine page type, an NLP topic extractor, a FastAPI layer to map failures to status codes, a CLI main function, and a pipeline to call the functions in sequence and return a JSON result. A Dockerfile containing seven instruction layers copies this code and its dependencies, and builds an image that AWS runs as a container to serve traffic.

Operationalizing Collection of Billions of URLs:

1. Requirements:

Workload

Inputs: Assume 1 billion URLs/month

Outputs: (per URL): HTML metadata, page type classification, ranked topics, stored raw content

Mode: No more interactive serving like in part 1. Now, a batch pipeline is optimized for throughput and monthly completion, rather than per-request latency. This batch process can be extended to continuous collection. The existing App Runner API remains as a lookup/debug side door, not the ingestion path.

Rationale for batch pipeline that accumulates returned metadata and content into a dataset for later querying: Using the App Runner API in part 1 to serve one billion URLs would take 60+ years, assuming each call waits 2 seconds to be served. Even with massive parallelization, a billion answers arriving over HTTP would be useless.

Assumptions/Estimations:

Sustained Crawl Rate: 1B URL Requests/30 days = 385 URLs/s. Traffic is never flat, so realistic peaks may be 3X or greater. Assume a max peak of 1500 URLs/s

385 URLs/s *(1.5s average fetch time) = roughly 600 in flight. At peak, 2250 in flight.

Raw HTML Volume: 1B*150 KB avg = 150 TB/month (30 TB/month after 5:1 zstd compression)

Metadata Volume: 1B*3KB = 3 TB/month

Compute Power: (1 parse takes ~75 ms). $1\text{B pages} \times 75\text{ms/page} = 75,000,000$ CPU-seconds / 3600 s/hr = 21,000 CPU-hours.

****Processing 1B pages takes around 21,000 hours of one CPU core's time (21K CPU-hours/month)****

$21,000 \text{ CPU-hours of work} / 730 \text{ hrs/month} = \text{roughly } 29 \text{ cores}$

Thus, we need 29 sustained vCPUs to parse 1B pages per month. 29 processor cores running 24/7 for one month would exactly finish parsing 1B pages.

29 cores rented on a couple of standard servers is on the order of a few hundred dollars a month. Totally feasible. The full production-grade project will not be limited by the cost of necessary computing power, even if the 1B parses/month doubles or even triples.

Soft Wall for the App Tier (I am currently using up some free credits on AWS and wouldn't wish to use more). The soft wall hit by the current minimum-cost setup is that it can only sustain roughly 660 URL crawl requests/s. This is estimated as follows:

Current uvicorn worker thread pool size ~ 40 workers, capped at 25 instances. Each worker spends about 1.5 s waiting for crawl results to come in. $40 \times 25 = 1000$ max crawls in flight at any time. $1000 \text{ workers busy} / 1.5\text{s} = 660$ completed crawls/s. We would be able to handle a sustained average of 385 URLs per second, but peak times would cause latency and crashing problems.

Domain Politeness Constraint:

It is assumed from the prompt that many of these requests may be directed at a small number of e-commerce platforms, such as Walmart and Amazon. All outbound fetches originate from a handful of AWS IP addresses. Simply paying for more MaxSize and instances to scale the AWS App Runner further will never solve the biggest problem - you can't simply send massive amounts of uncoordinated fetches to target sites without triggering safeguards against bot behavior and politeness controls. In testing this app, I already ran into difficulties fetching from Reddit and Amazon. Reddit returned a 403 to AWS-origin requests. Sending too many fetches to these sites will get the crawler IPs rate-limited and blacklisted. A responsible crawler deliberately limits how fast it hits each individual domain, to, say, no more than 100 requests/s.

At a per-domain limit of 100 requests/s: $1\text{B URLs} / 100 \text{ URLs/s} = 10,000,000 \text{ seconds} / 86,400 \text{ s/day} = \text{around } 115 \text{ days}$. The politeness math says it takes nearly four months to crawl 1B URLs on a single domain at a limit of 100 requests/s.

At 100 requests/s: $100 \text{ requests/s} \times 86,400 \text{ s/day} = 8.64\text{M URLs/day}$
 $8.64\text{M URLs/day} \times 25 \text{ days} = 216\text{M URLs}$

****Thus, a single domain can only be fully crawled in a month under our politeness constraint if it has fewer than 216M URLs****

Horizontal Scaling Won't Work:

Even if we made thousands of crawler machines, the limit is still the target. If Amazon only accepts 100 requests/s total before it bans the crawler, the thousands of machines don't increase speed; instead, they result in being banned more quickly.

2. Architecture:

Since we can't fetch faster, the system is built to fetch at a controlled rate, waste nothing, and never lose work. The pipeline has five stages:

1. Load the URLs: Read the monthly text file (or MySQL export), clean each URL (lowercase the domain, strip tracking junk like ?ref=), and drop duplicates. Every duplicate removed is the cost of a fetch saved. The cleaned URLs go into a waiting line.

2. The queue (waiting line): All URLs are queued, grouped by domain. This structure solves two problems. First, it controls how fast we release URLs to each domain, and second, it makes crashes harmless. If a cheap machine dies mid-fetch, its URL just goes back in the line, and another machine picks it up without losing anything. Use AWS Simple Queue Service (AWS SQS to start simply.)

3. Fetch workers: These pull URLs from the queue, but only as fast as the self-imposed politeness rate (assume 100 requests/second). All these workers do is download the raw HTML and save it to cheap bulk storage. They do not parse. Because the queue makes interruptions harmless, these can run on cheap interruptible machines. This is effectively `fetcher.py`, upgraded to handle many downloads at once.

4. Parse workers: These separate workers read the stored raw HTML and run the part 1 code: `parser.py`, `classifier.py`, and `topics.py` to produce the metadata, page type, and topics. This can be handled by 29 vCPUs as calculated earlier.

Fetching is the slow, rate-limited, expensive part; parsing is cheap. By storing the raw HTML once, I can improve the classifier later and re-run parsing on the stored HTML for a few hundred dollars of compute.

5. Storage:

- S3 compressed bulk storage for raw HTML (30 TB/month, but cheap and useful for re-parsing).
- Queryable table-format for the Final Metadata (3 TB/month or ~ 1 TB/month compressed, also on S3)

6. Politeness Controller Knob:

A Redis counter per domain can set the crawl speed, and it can be tested and audited per domain to dial up or down and optimize as necessary.

3. Unified Data Schema for Final Metadata:

Crawled data is stored in a single table, with one row per URL. Every URL produces a row with the same 35 columns, grouped into five categories: (excludes raw HTML stored in S3)

Identity and Provenance:

1. schema_version (text), format version
2. url_hash (text), primary key, a fingerprint of the URL
3. canonical_url (text), the cleaned URL
4. original_url (text), the raw URL from the input file
5. domain (text), the site
6. batch_id (text), which monthly batch, e.g., 2026-07
7. crawl_ts (timestamp), when it was fetched

Fetch Result:

8. fetch_outcome (text), ok / http_error / timeout / dns_failure / ssrf_blocked / non_html
9. fetch_status_code (int), HTTP status, e.g. 200 or 404
10. fetch_final_url (text), URL after redirects
11. fetch_elapsed_ms (int), how long the fetch took
12. fetch_bytes (int), bytes downloaded
13. fetch_truncated (bool), did it hit the 5 MB size cap?
14. content_type (text), e.g., text/html
15. content_ref (text), link to the saved raw HTML file

Page Metadata From the Parser:

16. page_title (text)
17. page_description (text)
18. page_keywords (list of text)
19. page_language (text)
20. page_author (text)

- 21. page_published_date (text)
- 22. page_word_count (int)
- 23. page_link_count (int)
- 24. page_image_count (int)
- 25. json_ld_types (list of text), structured-data types found
- 26. open_graph (JSON), the og: tags
- 27. headings (JSON), h1/h2/h3 lists
- 28. page_body_preview (text), first 500 characters of body

Classification:

- 29. page_type (text), product/news / bot_challenge / etc.
- 30. confidence (float), 0 to 1
- 31. classifier_signals (list of text), why it decided that
- 32. classifier_version (text), which classifier ran

Topics and Quality:

- 33. topics (list), each entry is {topic, score}
- 34. bot_challenged (bool), was this a bot wall?
- 35. parse_errors (list of text), any parse problems

4. SLOs and SLAs:

SLOs are the internal nonbinding targets, while SLAs are what is promised to the customer. In this case, I will consider monitoring the following metrics: Monthly batch delivered, fetch success rate, data completeness, lookup API availability, and data durability. Domains are considered in-ceiling only if their URL count, divided by their allowed crawl rate, is less than 25 days. For 100 requests/s, the URL limit is 216M. For domains above this number, polite crawling will not make a 30-day window feasible, and other delivery options must be negotiated.

Metric	SLO (internal target)	SLA (customer promise)
Monthly Batch Delivered	99% done within 25 days	99% of each within-ceiling domain done in 25 days, day-30 delivery for domains under the crawl-rate ceiling; higher-volume domains delivered on a negotiated schedule or via API
Fetch Success Rate	>= 85% per domain	>= 80% overall

Data Completeness	>= 99.9% good fetches fully parsed	>= 99.5%
Lookup API Availability	>= 99.9%	>= 99.5%
Data Durability	11 nines (S3)	8 nines

5. Monitoring, Metrics, and Tools:

While SLOs define the targets, monitoring is how progress towards them will be tracked in real-time. At the batch scale, the system will silently fall behind, so I would track metrics that reveal falling behind early, grouped into four questions:

1. Is the batch on track? Queue depth per domain and a projected completion date for each domain. Comparing the crawl rate to the queue size indicates whether it will finish by day 30, while there's still time to react (add machines to lagging domains or escalate to an API/licensing path).

2. Is what I'm fetching healthy? Fetch success vs. error rate per domain, and the bot-challenged rate. A spike in either means a site started blocking the app and needs its politeness rate dialed down or a different acquisition strategy.

3. Is the data good? Percentage of rows with a title/description filled in, and the classifier's confidence distribution. A sudden shift signals that a site has redesigned its pages, and the parser/classifier needs attention.

4. What is it costing? \$ per 1,000 URLs.

Tools I would Use: Amazon CloudWatch (or Prometheus + Grafana) to collect the metrics and build dashboards; An alerting tool like PagerDuty for the few problems worth an immediate alert, such as: projected completion slipping past deadline, a domain's success rate falling off a cliff, or cost forecast running over budget. Everything else is a ticket to review later.